2026 6th International Conference on Machine Learning and Intelligent Systems Engineering (MLISE) | 979-8-3315-8334-7/26/$31.00 ©2026 IEEE | DOI: 10.1109/MLISE70044.2026.11607496

Failure-Gated Hierarchical Memory: Preventing Memory Pollution in Long-Horizon LLM Agents via Critic-Guided Write Control

Qizhi Liu
National Science Library
Beijing, China
liuqizhi@mail.las.ac.cn

Meng Wang*
National Science Library
Beijing, China
wangmeng2022@mail.las.ac.cn

Jing Xie
National Science Library
Beijing, China
xiej@mail.las.ac.cn

***Abstract*-Large language model (LLM)-based agents equipped with persistent memory can accumulate experience across tasks, yet uncontrolled memory writes introduce memory pollution—erroneous, outdated, or over-generalized knowledge entries that silently degrade downstream task performance through retrieval-augmented generation. This phenomenon is particularly harmful in long-horizon deployments where the agent continuously operates over hundreds of episodes, as polluted entries compound over time via cascading error propagation. We present Failure-Gated Hierarchical Memory (FGHM), a three-tier memory architecture (working memory, episodic buffer, procedural long-term memory) augmented with a dedicated Failure Critic agent that gates all writes from the episodic buffer to procedural long-term memory. The Critic, implemented as a LoRA-tuned Llama-3-8B model trained on hard-negative knowledge candidates derived from failure trajectories, issues APPROVE, REJECT_REDUNDANT, or REJECT_CONFLICT verdicts for each candidate entry. Rejected-but-salvageable candidates undergo a reflective abstraction loop that progressively elevates the level of generalization. We further introduce sandbox consistency verification, where the Critic mentally replays historical failure scenarios with the candidate knowledge injected to detect potential downstream failures before commitment. Evaluated on ALFWorld (continuous 5-round reset, 670 scenarios), WebShop (200-task pipeline), and MemoryAgentBench FactConsolidation, FGHM reduces the long-term memory pollution rate from 38% (naive LLM summarization) to 9% while maintaining a task success rate degradation of only 4 percentage points over 5 rounds—compared to 20+ points for baselines. Comprehensive ablation studies confirm the independent contribution of each component. We additionally propose LTM Pollution Rate (LPR) as a new evaluation metric for memory quality assessment, complementing existing task-level metrics. Our results demonstrate that failure-aware write gating is essential for sustainable, long-horizon agent deployment.**

Keywords-failure-gated hierarchical memory (FGHM), large language model (LLM) agents, long-horizon agents, memory pollution, persistent memory.

I. INTRODUCTION

LLM-based autonomous agents increasingly rely on persistent memory to reuse experience across interactive tasks such as household decision making [1], web navigation [2], software engineering, and multi-step planning [3]. Persistent memory allows agents to accumulate evidence, retrieve prior solutions, and adapt across episodes rather than acting as isolated one-shot systems. Prior memory systems improve retrieval, context management, or reflective learning [4]–[9], enabling agents to generalize from successes and avoid repeated failures. However, most systems optimize how memories are retrieved, ranked, or forgotten rather than whether a candidate memory should be written in the first place.

A critical long-horizon failure mode is memory pollution: erroneous, stale, or over-generalized entries enter persistent storage and later contaminate retrieval-augmented reasoning. Pollution is especially harmful in continuous deployments because memory is repeatedly retrieved, reused, and updated. Because failed episodes may themselves generate new reflections, a single polluted entry can trigger cascading error propagation [10], gradually turning memory from a useful adaptation mechanism into a source of systematic bias. For example, an ALFWorld agent that repeatedly finds an apple on a countertop may store "apples are typically on the countertop." Retrieved later, this rule harms tasks where apples appear in a fridge or cabinet, and the resulting failures may produce further incorrect memories.

Existing memory approaches do not fully address this write-path problem. FIFO buffers discard entries indiscriminately, MemoryBank [11] relies on decay rather than correctness, and Reflexion [9] stores verbal self-critiques without independent verification. A-Mem [12] and related work likewise focus more on organization and retrieval than content-level write control.

We propose Failure-Gated Hierarchical Memory (FGHM), which adds an explicit quality gate between episodic experience and procedural long-term memory. Analogous to database commit checks, candidate knowledge is verified before it becomes persistent and retrievable. The framework keeps raw trajectories available for learning while ensuring that only validated, non-redundant, and non-conflicting rules can influence future episodes.

Our contributions are fourfold: (1) a three-tier memory architecture with a Failure Critic that gates L2→L3 writes through redundancy detection, conflict analysis, sandbox verification, and reflective abstraction; (2) a hard-negative construction pipeline from failure trajectories, followed by classification and DPO training for the Critic; (3) LTM Pollution Rate (LPR), a direct memory-quality metric; and (4) experiments on three benchmarks showing that FGHM reduces pollution by 4× while preserving long-horizon task performance.

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 16:09:49 UTC from IEEE Xplore. Restrictions apply.

II. Related Work

A. Memory Architectures for LLM Agents

Memory architectures for LLM agents have explored tiered storage, memory streams, decay-based retention, structured agentic memory, and production-oriented extraction [3]–[5], [8], [11]–[14]. MemGPT emphasizes virtual-memory-style context management, Generative Agents rank memory by recency, importance, and relevance, MemoryBank introduces forgetting curves based on classical memory decay theory [21], and HippoRAG explores neurobiologically inspired long-term memory for large language models [22]. These methods mainly improve indexing, retention, and retrieval priority; they do not explicitly verify the correctness or harmfulness of a candidate before it enters persistent memory.

B. Experience-Based Learning Agents

Experience-based agents such as Reflexion [9], ExpeL [6], and Voyager [7] learn from trajectories by storing reflections, insights, or reusable skills. They show that task feedback can improve future reasoning, but their write path is still permissive: useful and harmful reflections may be stored in the same way. In particular, most methods write the derived knowledge unconditionally or rely on the same primary LLM for both generation and judgment, leaving over-generalized reflections unchecked.

C. Memory Quality and Pollution

Recent studies directly expose this risk. Xiong et al. [10] show that indiscriminate memory addition propagates errors through experience-following behavior, while MemoryAgentBench [15] and MemBench [16] highlight difficulty in conflict resolution and memory quality evaluation. Recent work on very long-term conversational memory further shows the importance of evaluating memory systems over extended interactions rather than isolated short episodes [24]. These works motivate memory-quality metrics, but many remedies still operate after pollution has already entered the store. FGHM addresses the root cause by preventing harmful entries from being committed to long-term memory.

D. Preference Learning and Critique Models

Our Critic training draws on DPO [17] and LoRA [18]. DPO provides a stable preference-learning objective without explicit reward modeling, while LoRA enables efficient adaptation of an 8B Critic for frequent invocation. We adapt these tools to memory verification by pairing verified knowledge with plausible but flawed hard negatives, obtaining a lightweight Critic that can be invoked at every write attempt.

III. Preliminary

A. Problem Formulation

We consider an LLM-based agent A operating in an interactive environment ε over a sequence of task episodes $T = \{t_1, t_2, \ldots, t_N\}$. At each episode t_i, the agent receives a task specification, executes a sequence of actions based on its current policy (augmented by retrieved memory), and receives a binary outcome signal $o_i \in \{\text{success}, \text{failure}\}$. The agent maintains a memory system M that persists across episodes, influencing future behavior through retrieval-augmented generation[23].

Definition 1 (Memory Pollution). A long-term memory entry $m \in M$ is *polluted* if, when retrieved and injected into the agent's context for a future task, it increases the probability of task failure compared to not retrieving m. Formally, let $P(\text{fail} \mid m \in \text{context}) > P(\text{fail} \mid m \notin \text{context})$ define a polluted entry.

Definition 2 (Cascading Error). A cascading error occurs when a polluted memory entry m causes a task failure at episode t_j, and the agent's post-failure reflection generates a new entry m' that is also polluted, leading to further failures in subsequent episodes. The cascade depth d is the number of generations of polluted entries traceable to the original m.

The goal of FGHM is to minimize the LTM Pollution Rate while maintaining high task success rates, formally: min LPR(M) subject to $TSR(A, M) \geq TSR(A, M^*) - \varepsilon$, where M^* is the pollution-free memory and ε is a tolerance threshold.

B. Multi-Store Memory Model

Our memory hierarchy is inspired by the Atkinson-Shiffrin multi-store model [19] and Tulving's distinction between episodic and semantic memory [20]. We formalize three memory tiers with distinct retention periods, granularity levels, and write policies, as detailed in §4.1.

IV. Method: Failure-Gated Hierarchical Memory

FGHM is a write-controlled memory architecture for long-horizon LLM agents. The design principle is to decouple experience accumulation from durable knowledge commitment: raw trajectories may be stored freely, but procedural memory must pass verification. The framework combines a three-tier hierarchy (§4.1), a dedicated Failure Critic (§4.2), hard-negative Critic training (§4.3), and a reflective abstraction loop (§4.4).

Overview of the Data Flow. As shown in Figure 1, the agent executes each episode in L1 working memory, serializes the trajectory to L2 episodic memory after completion, then promotes only selected summaries to L3 procedural memory. The primary agent generates candidate knowledge from recent L2 evidence, but the Failure Critic independently evaluates that candidate against existing L3 contents and historical failures. Candidate L3 entries must pass redundancy detection, semantic conflict analysis, and counterfactual sandbox verification before becoming retrievable procedural knowledge.

A. Three-Tier Memory Hierarchy

FGHM separates memory into L1 working memory, L2 episodic buffer, and L3 procedural long-term memory, inspired by multi-store cognitive memory models [19], [20]. L1 is volatile and episode-scoped; L2 stores complete trajectories and failure attributions; L3 stores only compressed rules that have passed Critic review. This asymmetry lets the agent retain raw experience freely while protecting the persistent knowledge base from pollution, and it also creates a clean audit boundary between unverified evidence and verified knowledge.

L1: Working Memory. L1 contains dialogue history, observations, tool calls, and intermediate reasoning during one

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 16:09:49 UTC from IEEE Xplore. Restrictions apply.

task episode. It is implemented with LangGraph StateGraph checkpointing and is flushed to L2 when the episode ends.

$$\mathrm{L1}_t = \{msg_1, \ldots, msg_k\} \cup \{tool_1, \ldots, tool_j\} \cup \{obs_1, \ldots, obs_l\}$$

subject to the context window constraint $|\mathrm{L1}_t| \leq W$, where W is the model's maximum context length. L1 is ephemeral:

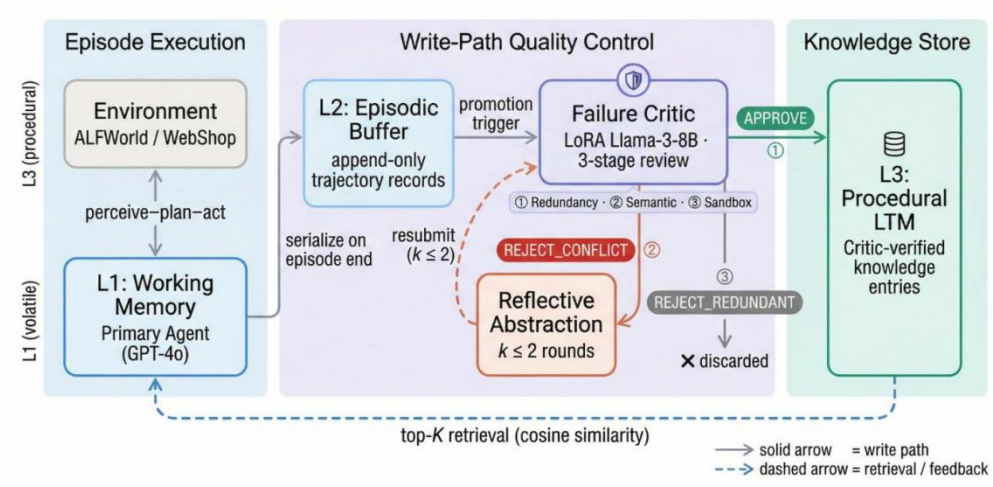


Figure 1: Overview of the FGHM framework.

upon task completion or failure, its contents are serialized into a structured trajectory record and flushed to L2, after which L1 is cleared for the next episode. This design ensures that L1 never contributes directly to long-term knowledge accumulation—it serves purely as a processing buffer.

L2: Episodic Buffer. L2 is an append-only vector store of trajectory records, each containing the serialized execution trace, outcome, failure attribution, retrieved L3 entry IDs, timestamp, and embedding. It supplies raw evidence for knowledge extraction and later sandbox checks.

$$r = (id,\ \text{task_id},\ \tau,\ o,\ f,\ \text{refs},\ t,\ \mathbf{e})$$

where τ is the full trajectory text (serialized from L1), $o \in$ {success,failure} is the binary outcome, f is an LLM-generated causal failure attribution (empty for successful episodes), *refs* $\subseteq$ L3 is the set of L3 entry IDs that were retrieved and injected into the agent's context during this episode, t is a timestamp, and $\mathbf{e} \in \mathbb{R}^d$ is the embedding vector computed by a sentence encoder (text-embedding-3-small). The *refs* field traces which L3 entries were active during a failure, providing direct evidence for sandbox verification (§4.2). L2 is append-only; records are never modified, with periodic garbage collection removing entries older than a configurable window (default: 30 days).

L3: Procedural Long-Term Memory. L3 contains immutable, generalizable knowledge rules with provenance metadata and abstraction-round counts. Updates are represented as superseding entries rather than in-place modifications, while conflicts are handled by the Critic.

$$m = (id,\ \text{text},\ \mathbf{e},\ \text{provenance},\ \text{timestamp},\ \text{version})$$

where *text* is a natural language knowledge statement, $\mathbf{e}$ is its embedding vector, *provenance* is the set of L2 record IDs from which the entry was derived, and *version* tracks how many reflective abstraction rounds were required before approval (0 for entries approved on first submission). L3 entries are immutable once written; updates are modeled as new entries that supersede older ones, with the Critic responsible for detecting and rejecting entries that conflict with existing knowledge.

To provide a unified view of memory lifecycle management across all three tiers, we summarize the write and eviction policies as follows. L1 (Working Memory) is written continuously at each agent timestep (perceive-plan-act cycle) and is fully cleared upon task completion or failure—its contents are serialized into a structured trajectory record and flushed to L2 before clearing. L2 (Episodic Buffer) is written automatically upon task completion via an append-only policy; entries older than the configurable garbage collection window (default: 30 days) are periodically evicted regardless of their content. L3 (Procedural Long-Term Memory) is written only when a candidate knowledge entry passes the Failure Critic's full multi-stage review; existing L3 entries are never modified in place—updates are modeled as new superseding entries. This asymmetry is intentional: L1 and L2 prioritize recall completeness by writing freely and evicting by time, while L3 prioritizes knowledge integrity by writing selectively and never overwriting verified entries.

At the start of each new task episode, the agent retrieves the top-K most relevant L3 entries by cosine similarity between the task description embedding and entry embeddings:

$$\text{Retrieved}(q) = \underset{m \in \mathrm{L3}}{\text{top-}K}\ \cos\left(\mathbf{e}_q, \mathbf{e}_m\right)$$

These entries are prepended to the agent's system prompt, providing persistent knowledge that guides action selection. The quality of L3 directly determines the value of this retrieval augmentation—polluted entries lead to polluted context, which leads to task failures.

Write Gate: L2→L3 Promotion. Promotion is triggered by buffer size, episode completion, a periodic timer, or explicit conflict detection. The primary agent proposes a candidate

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 16:09:49 UTC from IEEE Xplore. Restrictions apply.

summary from recent L2 records, but an independently trained Critic—not the generator—decides whether the entry may be committed.

The separation between candidate generation and candidate evaluation prevents the "grading your own homework" problem and makes the L2→L3 boundary the central control point for memory quality.

B. Failure Critic Agent

The Failure Critic is an independently trained gatekeeper for all L3 writes. Given a candidate entry and relevant existing L3 contents, it returns a verdict and reason, then triggers the corresponding downstream action.

Critic Interface. The verdict set is {APPROVE, REJECT_REDUNDANT, REJECT_CONFLICT}. APPROVE writes the entry to L3 with provenance; REJECT_REDUNDANT discards overlapping entries to prevent memory bloat; REJECT_CONFLICT sends risky but potentially useful candidates to reflective abstraction (§4.4). As illustrated in Figure 2, the Failure Critic follows a three-stage review pipeline consisting of redundancy detection, semantic conflict analysis, and sandbox consistency verification, followed by reflective abstraction when conflicts are detected.

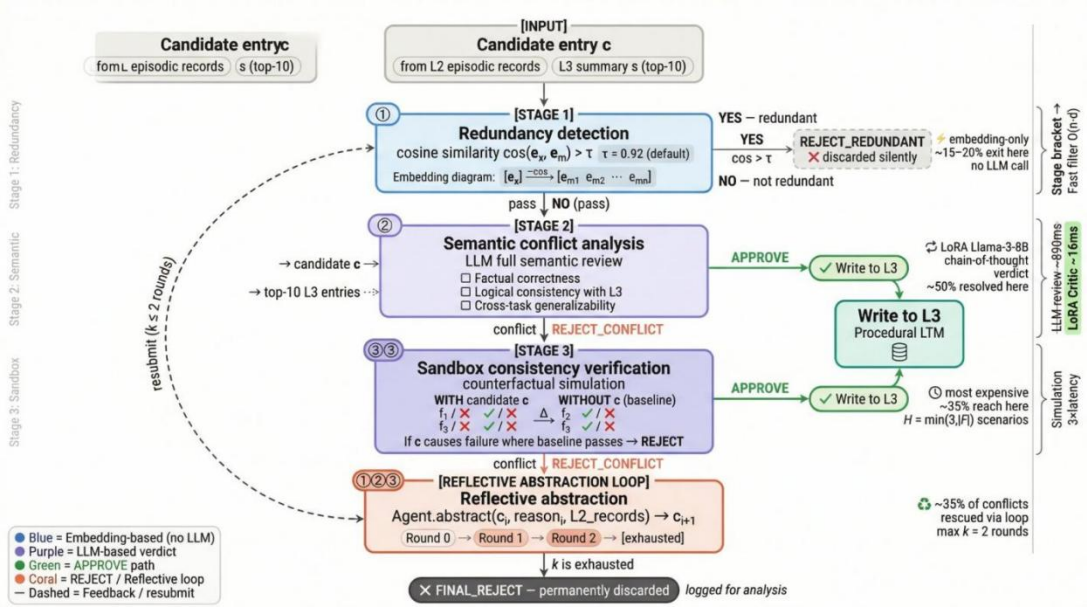


Figure 2: Three-stage review pipeline of the Failure Critic.

Stage 1: Redundancy Detection. The candidate embedding is compared with existing L3 embeddings. If cosine similarity exceeds $\tau = 0.92$, the entry is rejected as redundant without LLM inference.

$$\text{redundant}(c) = \exists\, m \in \text{L3}:\ \cos(\mathbf{e}_c, \mathbf{e}_m) > \tau$$

where τ is the redundancy threshold (default: 0.92). The value $\tau = 0.92$ was selected based on the empirical trade-off analysis in §5.8 (Figure 3c): at $\tau = 0.92$, the system achieves the optimal balance between pollution prevention (LPR = 9.2%) and knowledge retention (WGP = 83.6%); values above 0.92 begin to reject legitimately novel entries, reducing WGP without proportional LPR gain, while values below 0.90 allow semantically near-duplicate entries to pass, increasing memory bloat. If redundancy is detected, the Critic immediately issues REJECT_REDUNDANT without further analysis, saving computational resources. This stage filters approximately 15–20% of candidates in our experiments, providing a significant efficiency gain by avoiding unnecessary LLM inference.

Stage 2: Semantic Conflict Analysis. Candidates that pass redundancy checking are reviewed by the Critic against the top-10 relevant L3 entries for factual soundness, logical consistency, and transferable specificity. This stage identifies over-generalized rules, unsupported assumptions, and contradictions with already verified knowledge. The structured verdict and reason are logged for audit and possible abstraction.

Stage 3: Sandbox Consistency Verification. For candidates not immediately resolved in Stage 2, the Critic performs a counterfactual simulation we term *sandbox consistency verification*. Stage 3 is applied to all candidates that receive a REJECT_CONFLICT verdict in Stage 2, rather than only those with low Critic confidence. This design decision is motivated by our observation that text-level contradiction analysis (Stage 2) carries a false-positive rate of approximately 23% on subtle conflict cases in our validation set: some candidates that appear contradictory as text strings are in fact consistent when evaluated against concrete failure scenarios, and vice versa. Sandbox verification thus serves as a necessary second opinion that corrects Stage 2 errors in both directions—rescuing incorrectly flagged candidates and confirming genuinely harmful ones before the final verdict is issued. This stage is inspired by the observation that some knowledge entries appear correct in isolation but cause failures when combined with existing knowledge in specific task contexts—a subtle failure mode that pure text-level analysis would miss.

The Critic constructs an augmented knowledge set with the candidate injected, retrieves representative historical failure

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 16:09:49 UTC from IEEE Xplore. Restrictions apply.

scenarios, and simulates the action the agent would take under the augmented memory. The scenarios are selected from L2 cases whose failures involved related L3 entries, giving the sandbox concrete contexts rather than relying on isolated textual contradiction checks.

$$\hat{a}_i = \text{Critic.simulate}\big(K', \text{task}(f_i)\big), \quad \text{passed}_i = (\hat{a}_i = a_i^*)$$

A differential test then compares simulation with and without the candidate. If adding the candidate uniquely causes an incorrect action, the failure is attributed to that candidate and REJECT_CONFLICT is issued with scenario-level evidence.

Together, the fast embedding check, semantic review, and sandbox verification balance efficiency and detection accuracy: about 65% of candidates are resolved before the expensive sandbox stage, while 35% require counterfactual verification.

Implementation. The Critic is a LoRA-tuned Llama-3-8B-Instruct model with rank 16, α = 32, and dropout 0.1 on query/value projections. It reaches near-GPT-4o judgment quality at 16 ms per call on one A100, compared with 890 ms for GPT-4o (§5.7), making per-write invocation practical.

C. Hard-Negative Construction and Training

The Critic must recognize plausible but flawed knowledge rather than only obvious errors. Entries such as "countertops are primary fruit locations" may sound useful but encode an unsafe generalization. We therefore train the Critic with hard negatives derived from failure trajectories, where spurious but convincing rules are most likely to pollute memory.

Step 1: Error Rule Extraction. From failed L2 records, the primary LLM identifies the assumption or heuristic that directly caused failure, yielding concise error rules such as "Apples are always located on the countertop."

Step 2: Surface-Plausible Reformulation. Each error rule is rewritten into three authoritative-sounding variants that hide the flaw inside otherwise plausible context, creating difficult negative examples rather than trivial "always/never" failures. This prevents the Critic from learning only shallow lexical cues.

Step 3: Preference Dataset Assembly. The dataset contains APPROVE samples from successful trajectories (40%), REJECT_CONFLICT hard negatives (35%), and REJECT_REDUNDANT paraphrases of existing L3 entries (25%). This balance exposes the Critic to correct, conflicting, and redundant candidates. The final data include 3,200 labeled triples and 1,600 preference pairs.

Phase 1: Classification Pre-training. We first fine-tune the Critic with cross-entropy over APPROVE, REJECT_REDUNDANT, and REJECT_CONFLICT so that it learns the basic decision space for memory writes.

$$\mathcal{L}_{\text{CE}}(\theta) = -\sum_{i=1}^{|\mathcal{D}|} \log P_\theta(v_i \mid c_i, s_i)$$

The input contains an L3 summary, the candidate entry, and the instruction to produce a verdict with reasoning. Training uses AdamW, learning rate 2e-5, 10% warmup, batch size 8, and 3 epochs.

Phase 2: DPO Fine-tuning. We then apply DPO [17], using verified entries as chosen responses and flawed candidates as rejected responses, to sharpen the boundary between approval and rejection in ambiguous cases.

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\text{pref}}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

DPO runs for 2 epochs with learning rate 1e-5 over 1,600 preference pairs and improves the Critic's F1 by approximately 4 points over classification pre-training alone.

D. Reflective Abstraction Loop

A purely conservative Critic would discard useful partial insights. The reflective abstraction loop gives REJECT_CONFLICT candidates a bounded chance to be reformulated at a higher level of generality using the Critic's rejection reason and original L2 evidence.

This loop is motivated by cases where the specific candidate is wrong but the underlying lesson is useful. For example, "always check the cabinet first" may conflict with other object-specific rules, but it still suggests that search order should depend on object type and environment context. Without abstraction, such partially useful evidence would be lost; without gating, its brittle formulation would pollute L3.

$$c_{i+1} = \text{Agent.abstract}(c_i,\ reason_i,\ \text{L2_records}), \quad \text{for } i = 0,1,\dots,k-1$$

At each iteration, the agent removes situation-specific details and proposes a more transferable rule, which is resubmitted to the Critic. The loop stops when the candidate is approved, judged redundant, or reaches the retry budget k = 2; exhausted candidates are discarded and logged for future analysis.

Abstraction Quality. We observe a consistent pattern: each round increases generality while reducing specificity. For the cabinet-first example, the initial rejection "always check the cabinet first" is progressively abstracted over two rounds into the approved entry "Container search order should be determined by object type (heated items → microwave; perishables → fridge; utensils → drawer)"—not only non-conflicting but more actionable than the original.

In our experiments, approximately 35% of initially rejected candidates are successfully rescued through 1–2 rounds of reflective abstraction, contributing meaningfully to the agent's long-term knowledge base. This recovery rate demonstrates that the reflective loop is not merely a theoretical safeguard but a practically important mechanism for maximizing the yield of useful knowledge from agent experience.

V. EXPERIMENTS

A. Benchmarks and Evaluation Protocol

We evaluate FGHM on three complementary benchmarks that stress memory quality from different angles.

ALFWorld [1] is a text-based embodied environment with 134 household scenarios across six task types. We run 5

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 16:09:49 UTC from IEEE Xplore. Restrictions apply.

consecutive rounds without resetting memory, yielding 670 episodes and directly exposing pollution accumulation.

WebShop [2] is a simulated e-commerce environment where agents must find products matching natural language requirements. We use a 200-task sequential pipeline with evolving product information, creating opportunities for stale memories.

MemoryAgentBench [15] evaluates memory agents across retrieval, test-time learning, long-range understanding, and conflict resolution. We focus on FactConsolidation because it directly tests contradiction handling.

B. Baselines

We compare against five baselines spanning a spectrum of memory management strategies, from no memory to state-of-the-art reflection-based approaches.

No-Memory: The agent operates independently each episode with no persistent state, establishing a lower bound on experience utilization and an upper bound on robustness to pollution.

FIFO Buffer: A fixed-size sliding window (capacity: 50 entries) retaining the most recent memories, discarding older entries regardless of utility. This represents the simplest memory management strategy.

MemoryBank [11]: Applies Ebbinghaus forgetting curve-based decay, scoring memories by recency, access frequency, and importance. Memories below a decay threshold are pruned.

Naive LLM Summarization: The primary LLM directly compresses L2 episodic records into L3 entries without any Critic review. This is the direct write-without-gating paradigm that represents the default behavior in most memory-augmented agents.

Reflexion [9]: Generates verbal self-reflections after task failures and stores them unconditionally for future retrieval. Reflections are added to memory without verification, representing the state-of-the-art in reflection-based memory.

C. Metrics

We report TSR, ΔTSR = TSR@R1 − TSR@R5, LPR, WGP, CER, and FactConsolidation accuracy. TSR captures task-level performance, while ΔTSR captures long-horizon degradation. LPR is computed as the fraction of sampled L3 entries judged incorrect against ground-truth rules by GPT-4o; although scalable, this judge-based metric may inherit model bias, so we treat it as a complementary memory-quality measure rather than a replacement for task metrics. WGP measures rejection precision, and CER measures failures attributable to polluted retrieval.

D. Implementation Details

The primary agent is GPT-4o. The Failure Critic is LoRA-tuned Llama-3-8B-Instruct [25] with rank 16, $\alpha = 32$, and dropout 0.1 on query/value projections. L2 uses ChromaDB with text-embedding-3-small. Default settings are Nthresh = 50, $k = 2$, $\tau = 0.92$, and experiments run on 4×A100-80G GPUs, with Qwen2.5-72B-Instruct used for reproducibility checks.

E. Main Results (RQ1)

RQ1: Does FGHM maintain higher task performance and lower memory pollution than baselines over extended horizons?

TABLE 1: MAIN RESULTS ON ALFWORLD (CONTINUOUS 5-ROUND EVALUATION). ΔTSR = TSR@R1 − TSR@R5. ↓ LOWER IS BETTER, ↑ HIGHER IS BETTER.

Method	TSR @R1	TSR @R3	TSR @R5	ΔTSR (↓)	LPR (↓)	CER (↓)	WGP (↑)
No-Memory	46.3	46.3	46.3	0.0	N/A	N/A	N/A
FIFO Buffer	57.5	50.7	42.5	15.0	33.6	18.2	N/A
MemoryBank	61.2	55.2	49.3	11.9	27.4	14.8	N/A
Naive LLM Sum.	63.4	52.2	43.3	20.1	38.1	22.5	N/A
Reflexion	67.2	60.4	55.2	12.0	21.3	11.7	N/A
FGHM (Ours)	**68.7**	**67.2**	**65.0**	**3.7**	**9.2**	**4.1**	**83.6**

Table 1 shows that FGHM achieves the strongest long-horizon stability on ALFWorld: ΔTSR is only 3.7 points and LPR is 9.2%, whereas ungated memory methods degrade sharply.

Memory without gating hurts. FIFO Buffer and Naive LLM Summarization fall below the No-Memory baseline by R5, demonstrating that polluted memory can become worse than no memory at all.

Partial defenses are insufficient. MemoryBank and Reflexion reduce pollution relative to naive memory, but their LPR remains substantially higher than FGHM because they lack explicit write-path verification.

Gating prevents cascading errors. FGHM's CER is 4.1%, about 3× lower than Reflexion and 5× lower than Naive LLM Summarization, confirming that the gate breaks error propagation over time.

TABLE 2: RESULTS ON WEBSHOP AND MEMORYAGENTBENCH. FC = FACTCONSOLIDATION; AR = ACCURATE RETRIEVAL.

Method	WebShop Score (↑)	WebShop LPR (↓)	FC Single-Hop (↑)	FC Multi-Hop (↑)	AR Acc (↑)
No-Memory	42.1	N/A	32.4	2.1	44.7
FIFO Buffer	51.3	30.2	38.6	3.5	52.3
MemoryBank	55.6	25.8	41.2	4.2	58.1
Naive LLM Sum.	49.2	35.7	36.8	2.8	49.5
Reflexion	58.4	19.5	54.1	5.4	62.8
FGHM (Ours)	**71.8**	**11.4**	**67.3**	**8.9**	**73.2**

Table 2 shows the same pattern beyond ALFWorld. On WebShop, FGHM reaches 71.8% score with 11.4% LPR, outperforming Reflexion under evolving product information. On MemoryAgentBench, FGHM improves FactConsolidation single-hop accuracy to 67.3% and multi-hop accuracy to 8.9%, though multi-hop conflict resolution remains difficult for all methods.

F. Ablation Study (RQ2)

RQ2: What is the independent contribution of each FGHM component?

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 16:09:49 UTC from IEEE Xplore. Restrictions apply.

TABLE 3: ABLATION STUDY ON ALFWORLD (R5). EACH ROW REMOVES ONE COMPONENT FROM FGHM.

Variant	TSR@R5 (↑)	ΔTSR (↓)	LPR (↓)	WGP (↑)	CER (↓)
FGHM (Full)	**65.0**	**3.7**	**9.2**	**83.6**	**4.1**
– Critic	47.8	18.5	34.6	N/A	19.8
– Sandbox	58.4	8.2	17.4	76.3	9.6
– Reflection	60.1	6.8	13.1	71.2	7.4
– HardNeg	57.2	9.4	15.8	73.8	8.9

Table 3 confirms the contribution of each component. Removing the Critic causes the largest collapse (TSR@R5 65.0%→47.8%; LPR 9.2%→34.6%), showing that FGHM without write gating degenerates into ordinary summarization. Removing sandbox verification raises LPR to 17.4% by missing context-dependent conflicts that are not obvious from textual contradiction alone. Removing reflection lowers WGP by discarding recoverable knowledge; training without hard negatives raises LPR to 15.8% by weakening detection of plausible-but-flawed candidates.

G. Critic Implementation Analysis (RQ3)

RQ3: Is a specialized fine-tuned Critic necessary, or can general-purpose LLMs serve the same role?

TABLE 4: COMPARISON OF CRITIC IMPLEMENTATIONS. LATENCY MEASURED PER SINGLE EVALUATION.

Critic Type	WGP (↑)	Recall (↑)	F1 (↑)	Latency (ms/call)
LoRA-Critic (Ours)	**83.6**	**79.1**	**81.3**	**16**
GPT-4o Zero-Shot	86.2	82.4	84.3	890
Rule-Based	91.3	34.6	50.2	2

Table 4 compares Critic implementations. The LoRA-Critic approaches GPT-4o zero-shot accuracy while reducing latency from 890 ms to 16 ms per evaluation, making it suitable for every write attempt. A rule-based Critic is fast and precise but has only 34.6% recall, missing most subtle pollution cases because harmful entries often lack obvious keywords.

H. Hyperparameter Sensitivity (RQ4)

We analyze sensitivity to three key hyperparameters on ALFWorld.

L2 promotion threshold N_{thresh}. Values of 20, 50, and 100 yield TSR@R5 of 62.4%, 65.0%, and 63.8% respectively (Figure 3a). Too-small thresholds trigger frequent but noisy write attempts with insufficient evidence for accurate summarization. Too-large thresholds delay knowledge consolidation, preventing the agent from benefiting from recent experiences. The optimal value (N_{thresh} = 50) balances write quality with knowledge currency. This result directly justifies the default Nthresh = 50 used in all main experiments.

Maximum reflection rounds k. k=0 (no reflection) reduces TSR@R5 to 60.1% (Figure 3b), confirming the loop's value. k=1 yields 63.2%; k=2 achieves 65.0%; k=3 provides negligible further improvement (65.3%) at 1.5× the write latency. Diminishing returns beyond k=2 suggest that entries requiring more than two abstraction rounds are unlikely to contain recoverable useful knowledge. Based on this trade-off between recovery rate and latency, k = 2 is selected as the default, providing maximum recovery yield without incurring the additional cost of a third round.

Redundancy threshold τ. As shown in Figure 3c, τ values of 0.85, 0.90, 0.92, and 0.95 yield LPRs of 14.2%, 11.1%, 9.2%, and 7.8%, while WGP decreases from 87.1% to 83.6% to 78.4% as τ increases due to increasingly aggressive filtering that begins rejecting legitimately novel entries. The default τ = 0.92 provides an effective balance between pollution prevention and knowledge retention. This analysis constitutes the quantitative basis for the τ = 0.92 default, which was not selected ad hoc but determined by the inflection point of the LPR-WGP trade-off curve in Figure 3c.

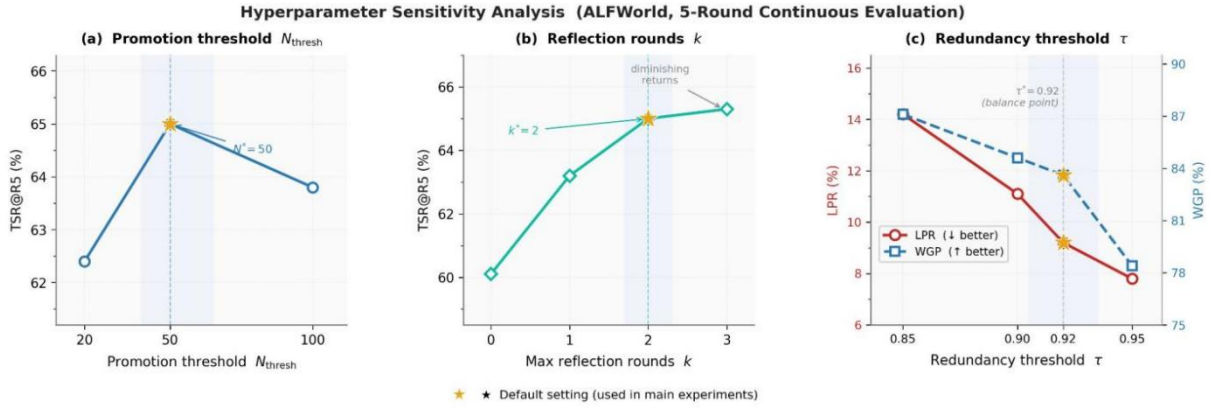


Figure 3: Hyperparameter sensitivity on ALFWorld (5-round continuous evaluation). ★ marks the default setting used in all main experiments. (a) TSR@R5 peaks at N_thresh = 50; smaller values yield noisy candidates while larger values delay consolidation. (b) Performance saturates at k = 2 with negligible gain at k = 3. (c) LPR and WGP exhibit an inherent trade-off as τ increases; τ = 0.92 achieves the optimal balance between pollution prevention and knowledge retention.

VI. CONCLUSION

We have presented Failure-Gated Hierarchical Memory (FGHM), a framework that addresses memory pollution in long-horizon LLM agents through failure-aware gating of long-term memory writes. By training a specialized Failure Critic on hard negatives derived from failure trajectories, employing sandbox consistency verification, and implementing a reflective abstraction loop for knowledge recovery, FGHM reduces memory pollution rates by 4× compared to naive summarization while maintaining stable task performance across extended

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 16:09:49 UTC from IEEE Xplore. Restrictions apply.

deployment horizons. The introduction of the LTM Pollution Rate metric provides the community with a direct, interpretable measure of memory quality that complements existing task-level metrics.

Our results carry a clear practical message: for any production deployment of memory-augmented LLM agents, explicit write-path quality control is not merely beneficial—it is necessary. Without gating, agents can degrade below memoryless baselines within just 5 rounds of continuous operation. We believe FGHM's modular design—where the Critic operates as an independent, pluggable component—facilitates adoption across diverse agent architectures and deployment settings. Future work should explore adaptive Critic training that continuously updates from in-deployment failures, as well as extensions to multi-agent settings where memory pollution can propagate across agent boundaries.

Our work also has several limitations. The Failure Critic still misses a portion of erroneous entries, especially subtle domain-specific errors, and may require retraining or adaptation under new task distributions. Sandbox verification introduces additional latency, which can limit scalability in high-throughput settings. Moreover, our evaluation is still conducted in a simulated environment, the LPR metric depends on an LLM judge and thus inherits potential annotation bias. More fundamentally, LLM-based evaluation of factual correctness is bounded by the judge's own domain knowledge: for task environments outside the training distribution of the judge model, LPR estimates may be systematically biased. Future work should explore calibrated human-in-the-loop evaluation protocols that use human annotation to anchor LLM judgments on a representative subset, and the current framework is limited to text-based memory rather than multimodal settings.

REFERENCES

[1] Shridhar, M., Thomason, J., Gordon, D., et al. (2021). ALFWorld: Aligning text and embodied environments for interactive learning. In Proceedings of ICLR 2021.

[2] Yao, S., Chen, H., Yang, J., and Narasimhan, K. (2022). WebShop: Towards scalable real-world web interaction with grounded language agents. In Advances in NeurIPS 35.

[3] Wang, L., Ma, C., Feng, X., et al. (2024a). A survey on large language model based autonomous agents. Frontiers of Computer Science, 18(6):186345.

[4] Zhang, Z., Dai, Q., Chen, X., et al. (2024). A survey on the memory mechanism of large language model-based agents. ACM Transactions on Information Systems.

[5] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., and Gonzalez, J. E. (2024). MemGPT: Towards LLMs as operating systems. In Advances in NeurIPS 36.

[6] Zhao, A., Huang, D., Xu, Q., Lin, M., Liu, Y., and Huang, G. (2024). ExpeL: LLM agents are experiential learners. In Proceedings of AAAI, volume 38.

[7] Wang, G., Xie, Y., Jiang, Y., et al. (2023). Voyager: An open-ended embodied agent with large language models. In Advances in NeurIPS 36.

[8] Li Y, Wang M, Zhang M, et al. TrustSciAgent: Towards Rigorous and Trustworthy Agents for Scientific Research[C]//2025 IEEE 24th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom). IEEE, 2025: 3207-3214.

[9] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. (2024). Reflexion: Language agents with verbal reinforcement learning. In Advances in NeurIPS 36.

[10] Xiong, R., Li, Y., Xiao, Z., and Chen, Y. (2025). How memory management impacts LLM agents: An empirical study of experience-following behavior. arXiv preprint arXiv:2505.16067.

[11] Zhong, W., Guo, L., Gao, Q., Ye, H., and Wang, Y. (2024). MemoryBank: Enhancing large language models with long-term memory. In Proceedings of AAAI, volume 38.

[12] Xu, Z., Peng, B., Xie, P., and Huang, X. (2025). A-Mem: Agentic memory for LLM agents. arXiv preprint arXiv:2502.12110.

[13] Sumers, T. R., Yao, S., Narasimhan, K., and Griffiths, T. L. (2024). Cognitive architectures for language agents. Transactions on Machine Learning Research.

[14] Chhikara, P., Khare, S., Arora, D., and Deshpande, T. (2025). Mem0: Building production-ready AI agent memory. arXiv preprint arXiv:2504.19413.

[15] Hu, Y., Wang, Y., and McAuley, J. (2025). Evaluating memory in LLM agents via incremental multi-turn interactions. In Proceedings of ICLR 2026.

[16] Tan, Z., Wu, Y., Chen, J., et al. (2025). MemBench: Towards more comprehensive evaluation on the memory of LLM-based agents. arXiv preprint arXiv:2506.21605.

[17] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. In Advances in NeurIPS 36.

[18] Hu, E. J., Shen, Y., Wallis, P., et al. (2022). LoRA: Low-rank adaptation of large language models. In Proceedings of ICLR 2022.

[19] Atkinson, R. C. and Shiffrin, R. M. (1968). Human memory: A proposed system and its control processes. In Psychology of Learning and Motivation, volume 2, pages 89–195. Academic Press.

[20] Tulving, E. (1972). Episodic and semantic memory. In Organization of Memory, pages 381–403. Academic Press.

[21] Ebbinghaus, H. (1885). Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie. Duncker & Humblot.

[22] Gutiérrez, B. J., Shu, Y., Gu, Y., Yasunaga, M., and Su, Y. (2024). HippoRAG: Neurobiologically inspired long-term memory for large language models. In Advances in NeurIPS 37.

[23] Lewis, P., Perez, E., Piktus, A., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In Advances in NeurIPS 33.

[24] Maharana, A., Lee, D.-H., Tulyakov, S., Bansal, M., Barbieri, F., and Fang, Y. (2024). Evaluating very long-term conversational memory of LLM agents. In Proceedings of ACL 2024.

[25] Meta AI (2024). Llama 3 model card. https://github.com/meta-llama/llama3.

Authorized licensed use limited to: University of Kansas Libraries. Downloaded on October 13,2026 at 16:09:49 UTC from IEEE Xplore. Restrictions apply.